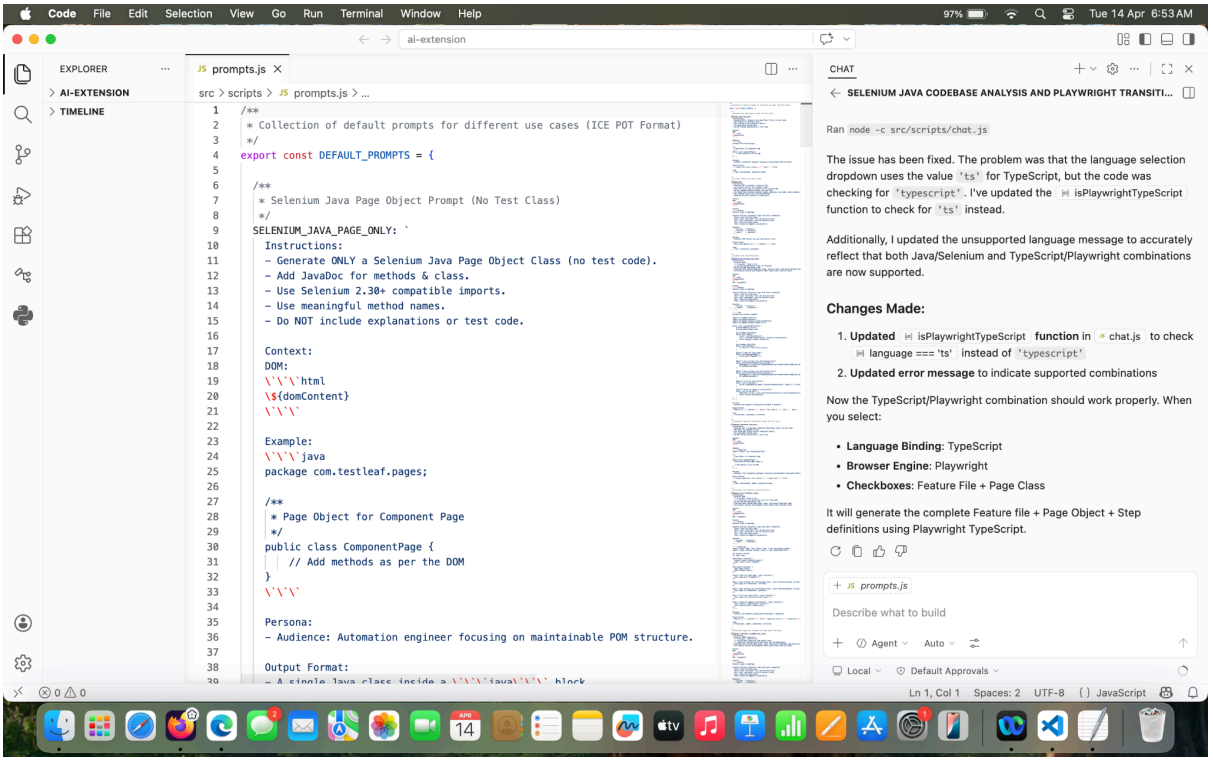
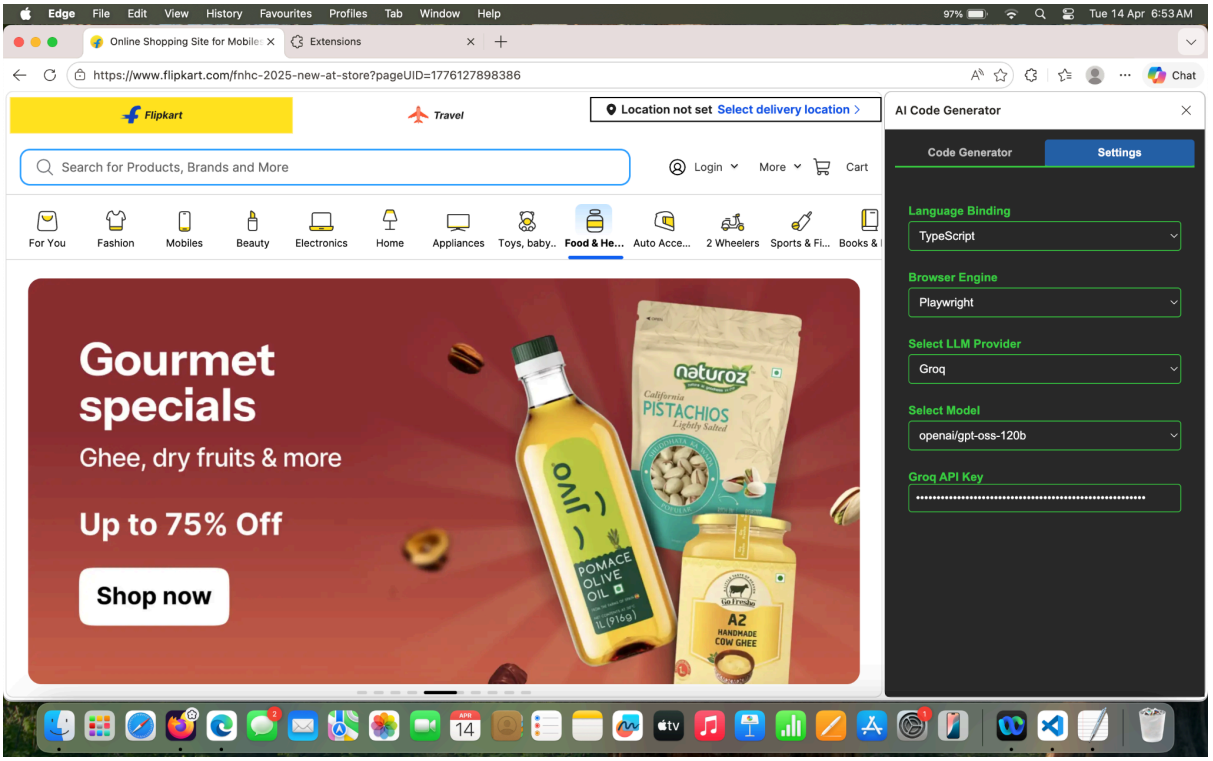
Query :

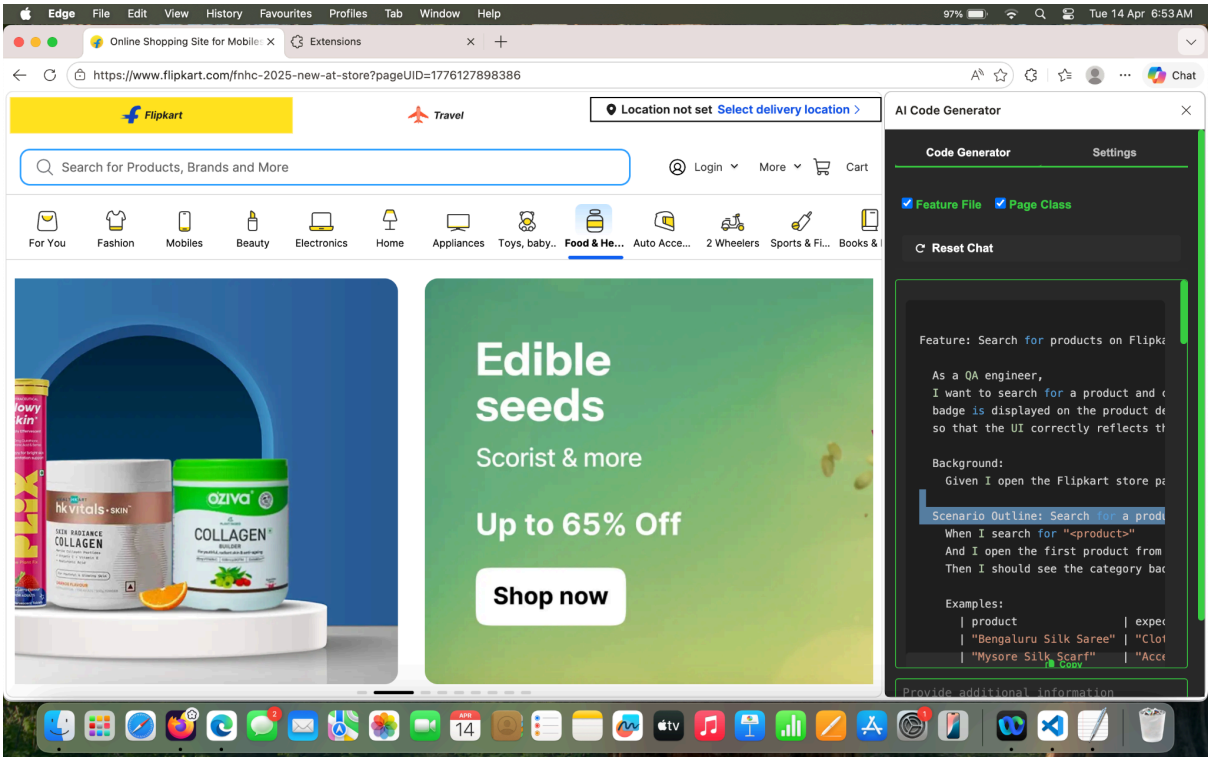
Analyze my entire code base for selenium java (Feature file,Page class,Fature+ Page class)

GO STEP BY STEP .ONCE UP ON MY APPROVAL PROCEED WITH NEXT STEP

--Give me the files in which implemenation is done
--Proceed with Playwright Typescript implementation







Output :

```
Feature: Search for products on Flipkart and verify their category label
```

```
  As a QA engineer,
  I want to search for a product and confirm that the correct category
  badge is displayed on the product details page,
  so that the UI correctly reflects the product classification.

  Background:
    Given I open the Flipkart store page

  Scenario Outline: Search for a product and validate its category badge
    When I search for "<product>"
    And I open the first product from the results
    Then I should see the category badge containing "<expectedCategory>"

    Examples:
      | product                | expectedCategory |
      | "Bengaluru Silk Saree" | "Clothing, Women"|
      | "Mysore Silk Scarf"    | "Accessories"    |
      | "Coorg Coffee Beans"   | "Grocery, Food"  |
```

```
import { Page, Locator, expect } from '@playwright/test';

/**
 * Page Object representing the Flipkart store page and its search flow.
 */
export class FlipkartStorePage {
  private readonly url =

'https://www.flipkart.com/fnhc-2025-new-at-store?pageUID=1776127898386';
  private readonly searchInput: Locator;
  private readonly searchButton: Locator;
  private readonly productResults: Locator;
  private readonly categoryBadge: Locator;

  constructor(private readonly page: Page) {
    // Search bar (Flipkart uses an input with name="q" on the header)
    this.searchInput = this.page.locator('input[name="q"]');
    this.searchButton = this.page.locator('button[type="submit"]');

    // First product tile in the grid of results
    this.productResults = this.page.locator(
      'div[data-id] >> nth=0' // generic selector for the first result
    );

    // Category badge - the snippet you provided uses a very specific class
list.
    // We'll locate it by its unique class pattern and visible text.
    this.categoryBadge = this.page.locator(

'div.css-146c3p1.r-dnmrzs.r-1udh08x.r-1udbk01.r-3s2u2q.r-1iln25a.r-1b1g84l.
r-tuq35u.r-u8s1d.r-q4m81j.r-1r0gou5'
    );
```

```
  }

  /** Navigate to the store page */
  async openStorePage(): Promise<void> {
    await this.page.goto(this.url, { waitUntil: 'domcontentloaded' });
    await this.page.waitForLoadState('networkidle');
  }

  /** Type a product name into the global search box */
  async searchProduct(productName: string): Promise<void> {
    await this.searchInput.waitFor({ state: 'visible' });
    await this.searchInput.fill(productName);
    await this.searchButton.click();
    // Wait for the results grid to appear
    await this.page.waitForSelector('div[data-id]', { state: 'visible' });
  }

  /** Click the first product from the search results */
  async openFirstResult(): Promise<void> {
    await this.productResults.waitFor({ state: 'visible' });
    await this.productResults.click();
    // Switch to the newly opened product detail page (may be same tab)
    await this.page.waitForLoadState('domcontentloaded');
  }

  /** Verify that the category badge contains the expected text */
  async verifyCategoryBadge(expectedText: string): Promise<void> {
    await this.categoryBadge.waitFor({ state: 'visible' });
    const badgeText = await this.categoryBadge.textContent();
    expect(badgeText?.trim()).toContain(expectedText);
  }
}
```

```
import { Before, After, Given, When, Then } from '@cucumber/cucumber';
import { chromium, Browser, Page } from '@playwright/test';
import { FlipkartStorePage } from './FlipkartStorePage';

let browser: Browser;
let page: Page;
let storePage: FlipkartStorePage;

/* ---------- Hooks ---------- */
Before(async function
  } () {
  // Launch a headless Chromium instance; you can toggle headless via env
var if needed
  browser = await chromium.launch({ headless: true });
  const context = await browser.newContext();
  page = await context.newPage();
  storePage = new FlipkartStorePage(page);
```

```
});

After(async function () {
  await page.close();
  await browser.close();
});

/* ---------- Step Definitions ---------- */
Given('I open the Flipkart store page', async function () {
  await storePage.openStorePage();
});

When('I search for {string}', async function (product: string) {
  await storePage.searchProduct(product);
});

When('I open the first product from the results', async function () {
  await storePage.openFirstResult();
});

Then(
  'I should see the category badge containing {string}',
  async function (expectedCategory: string) {
    await storePage.verifyCategoryBadge(expectedCategory);
);
```